In [ ]:

```
#Name: Meenakshi Sangeeth
#Registration No: 240905630
#Section: PG
#Roll No: 34
```

In [ ]:

```
#Lab Manual
```

In [11]:

```
#1.Generate a 3x4 NumPy array with random integers between 1 and 50.
#a. Calculate and print the Mean, Median, and Standard Deviation of the
#b. Print the Sum of all elements and the sum of each row.
#c. Reshape the 3x4 array into a 2x6 array and print it.

import numpy as np

a = np.random.randint(1,50,12)
a = a.reshape(3,4)
print("Matrix: ")
print(a)

print("Mean: ", np.mean(a))
print("Median: ", np.median(a))
print("Standad Deviation", np.std(a))

print('Sum of all elements: ', np.sum(a))
x=np.sum(a, axis=1)
for i in range (3):
    print('Sum of row',i+1,":",x[i])

print("2x6 matrix: ")
print(a.reshape(2,6))


```

```
Matrix:
[[ 1  3 30  3]
 [30 45 30 13]
 [42  1 14 47]]
Mean:  21.583333333333332
Median:  22.0
Standad Deviation 17.06096096030024
Sum of all elements:  259
Sum of row 1 : 37
Sum of row 2 : 118
Sum of row 3 : 104
2x6 matrix:
[[ 1  3 30  3 30 45]
 [30 13 42  1 14 47]]
```

In [17]:

```python
#2. Create two (3 * 3) matrices using NumPy and print it. Perform and pr
#a. Matrix addition
#b. Matrix subtraction
#c. Matrix multiplication (element-wise and dot product)
#d. Transpose of a matrix
#e. Determinant and inverse (if applicable)

import numpy as np

a1 = np.random.randint(11,20,9)
a1 = a1.reshape(3,3)
print("Matrix 1:\n",a1)

a2 = np.random.randint(1,10,9)
a2 = a2.reshape(3,3)
print("Matrix 2:\n",a2)

#a. Matrix addition
print("Matrix sum:")
print(a1+a2)

#b. Matrix subtraction
print("Matrix difference:")
print(a1-a2)

#c. Matrix multipliation and dot product
print("Matrix multiplication:")
print(a1*a2)
print("Dot product:")
print(np.dot(a1,a2))

#d. transpose of a matrix
print("Transpose of matrix 1: ")
print(a1.T)
print("Transpose of matrix 2: ")
print(a2.T)

#e. determinant and inverse
print("Determinant of matrix 1:", np.linalg.det(a1))
print("Determinant of matrix 2:",np.linalg.det(a2))
print("Inverse of matrix 1:\n",np.linalg.inv(a1))
print("Inverse of matrix 2:\n",np.linalg.inv(a2))
```

```
Matrix 1:
 [[12 14 15]
 [19 17 12]
 [17 11 14]]
Matrix 2:
 [[7 8 2]
 [1 1 3]
 [5 2 5]]
Matrix sum:
[[19 22 17]
 [20 18 15]
 [22 13 19]]
Matrix difference:
[[ 5  6 13]
 [18 16  9]
 [12  9  9]]
Matrix multiplication:
[[ 84 112  30]
 [ 19  17  36]
 [ 85  22  70]]
Dot product:
[[173 140 141]
 [210 193 149]
 [200 175 137]]
Transpose of matrix 1:
[[12 19 17]
 [14 17 11]
 [15 12 14]]
Transpose of matrix 2:
[[7 1 5]
 [8 1 2]
 [2 3 5]]
Determinant of matrix 1: -796.0
Determinant of matrix 2: 66.99999999999997
Inverse of matrix 1:
 [[-0.13316583  0.03894472  0.10929648]
 [ 0.07788945  0.10929648 -0.17713568]
 [ 0.10050251 -0.13316583  0.07788945]]
Inverse of matrix 2:
 [[-0.01492537 -0.53731343  0.32835821]
 [ 0.14925373  0.37313433 -0.28358209]
 [-0.04477612  0.3880597  -0.01492537]]
```

In [ ]:

```
#Additional
```

In [51]:

```python
#1. Given the array arr = np.arange(1, 101).reshape(10, 10):
#a) Extract the first three rows and the last three columns.
#b) Extract the diagonal elements.
import numpy as np
arr = np.arange(1, 101).reshape(10, 10)

print("First 3 rows:")
print(arr[:3])
print("Last 3 columns:")
print(arr[:,-3:])

print("Diagonal elements: ")
print(np.diag(arr))
```

```
First 3 rows:
[[ 1  2  3  4  5  6  7  8  9 10]
 [11 12 13 14 15 16 17 18 19 20]
 [21 22 23 24 25 26 27 28 29 30]]
Last 3 columns:
[[  8   9  10]
 [ 18  19  20]
 [ 28  29  30]
 [ 38  39  40]
 [ 48  49  50]
 [ 58  59  60]
 [ 68  69  70]
 [ 78  79  80]
 [ 88  89  90]
 [ 98  99 100]]
Diagonal elements:
[  1  12  23  34  45  56  67  78  89 100]
```

In [68]:

```python
#2.Create a 3D NumPy array of shape (4, 5, 6) with random integers from
# Extract all elements from the second row of every 2D matrix along the
# Extract all elements from the last column of the third 2D matrix along
import numpy as np

a = np.random.randint(1, 100,120)
a = a.reshape(4,5,6)
print("Matrix:\n",a)

print("\nElements from the second row of every 2D matrix along the first
print(a[:, 1:2, :1])

print("\nElements from the last column of the third 2D matrix along the
print(a[2:3, 0:1, 5:])
```

```
Matrix:
 [[[13 48 40 69 57 19]
  [53 30 77 53 13 29]
  [25 54 25  5 84 44]
  [46 19 39 44 97 50]
  [71 36  4 16 98 73]]

 [[41 55 58 52 42 19]
  [35 86 33 70 64 95]
  [67  8 40 17 30 91]
  [ 2 25 32 57 82 78]
  [47 40 68  2 60 98]]

 [[83 53 63 71 62 43]
  [30 84 56 50 15 82]
  [48 71 97 54 48 56]
  [10 78 70 98 70  4]
  [96  2 77 81 17 81]]

 [[87 80 96 74 54 88]
  [46 34 83 74 65 27]
  [81  2 83  9 22 73]
  [60 69 96 61 85 26]
  [60 45 65  3 80 36]]]

Elements from the second row of every 2D matrix along the first axis:
[[[53]]

 [[35]]

 [[30]]

 [[46]]]

Elements from the last column of the third 2D matrix along the first axis:
[[[43]]]
```

In [71]:

```python
#3. Create a 4D array of shape (3, 4, 5, 6) with random floating-point r
# Extract every alternate row and column from the second and fourth dime
# Extract elements from every 2nd matrix along the first axis and every
import numpy as np
a = np.random.rand(3,4,5,6)
print("Matrix:\n",a)

print("\nAlternate row and column from the second and fourth dimensions
print(a[:, ::2, :, ::2])

print("Elements from every 2nd matrix along the first axis and every 3r
print(a[::2, :, ::3, :])
```

```
Matrix:
 [[[[0.61152025 0.60650008 0.16681404 0.03644009 0.34232774 0.30228037]
   [0.49888013 0.48581519 0.38317121 0.44350672 0.57790905 0.42642013]
   [0.02338092 0.93572414 0.40397315 0.91462677 0.23477616 0.50863082]
   [0.17859726 0.85116563 0.9076469  0.44142243 0.82486789 0.02096496]
   [0.60124162 0.73909052 0.53535183 0.20058165 0.6110353  0.8651816 ]]

  [[0.49972462 0.39427697 0.45450578 0.83966235 0.39571265 0.62829774]
   [0.34069228 0.84465757 0.05910338 0.13876458 0.19801074 0.2592925 ]
   [0.39499505 0.63188548 0.55669152 0.82548463 0.36636708 0.71489976]
   [0.69527001 0.24066892 0.47586155 0.43980771 0.88743461 0.67556747]
   [0.47838076 0.95865122 0.44262973 0.30238665 0.33813354 0.3986594 ]]

  [[0.07861903 0.26004921 0.58808257 0.96762072 0.57824504 0.97110291]
   [0.20874145 0.39756707 0.02975713 0.22172295 0.12737327 0.15757309]
   [0.32044366 0.40830145 0.43351619 0.0642167  0.38787255 0.53159243]
   [0.63542053 0.29518766 0.82909971 0.34901518 0.42845579 0.98664267]
   [0.14801347 0.55842011 0.68159756 0.57284496 0.91941397 0.81823996]]
```

In [34]:

```python
#4. Create a 2D array of shape (6, 8) with numbers from 1 to 48.
#Reverse the rows while keeping the columns in their original order.
#Reverse the last three columns while keeping the rows in their origina

a = np.arange(1, 49)
a = a.reshape(6, 8)
print("Matrix:\n", a)

rev = a[::-1]
print('\nMatrix after eversing rows:')
print(rev)

print('\nMatrix after reversing the last three columns:')
a[:,-3:] = a[:,-3:][:,::-1]
print(a)

```

```
Matrix:
 [[ 1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16]
 [17 18 19 20 21 22 23 24]
 [25 26 27 28 29 30 31 32]
 [33 34 35 36 37 38 39 40]
 [41 42 43 44 45 46 47 48]]

Matrix after eversing rows:
[[41 42 43 44 45 46 47 48]
 [33 34 35 36 37 38 39 40]
 [25 26 27 28 29 30 31 32]
 [17 18 19 20 21 22 23 24]
 [ 9 10 11 12 13 14 15 16]
 [ 1  2  3  4  5  6  7  8]]

Matrix after reversing the last three columns:
[[ 1  2  3  4  5  8  7  6]
 [ 9 10 11 12 13 16 15 14]
 [17 18 19 20 21 24 23 22]
 [25 26 27 28 29 32 31 30]
 [33 34 35 36 37 40 39 38]
 [41 42 43 44 45 48 47 46]]
```

In [53]:

```python
#5. Create a 3D array of shape (3, 5, 5) with random integers from 10 t
#Extract all elements from the array where the value is a multiple of 3
#Replace all elements divisible by 5 with the value 0.
import numpy as np

a = np.random.randint(10, 100,75)
a = a.reshape(3,5,5)
print("Matrix:\n",a)

print("\nElements which are multiples of 3:\n", a[a%3==0])

a[a%5==0] = 0
print('\nMatrix after replacing elements divisible by 5:\n',a)

```

```
Matrix:
 [[[34 25 48 71 97]
  [75 96 52 90 46]
  [93 53 96 40 93]
  [48 45 44 65 90]
  [62 52 46 76 68]]

 [[55 81 47 24 95]
  [15 68 87 68 83]
  [86 80 38 94 17]
  [30 52 81 99 59]
  [37 69 55 66 68]]

 [[26 81 72 13 92]
  [40 73 67 49 21]
  [13 18 36 26 79]
  [66 36 30 12 92]
  [95 20 48 44 43]]]

Elements which are multiples of 3:
 [48 75 96 90 93 96 93 48 45 90 81 24 15 87 30 81 99 69 66 81 72 21 18 36
 66 36 30 12 48]

Matrix after replacing elements divisible by 5:
 [[[34  0 48 71 97]
  [ 0 96 52  0 46]
  [93 53 96  0 93]
  [48  0 44  0  0]
  [62 52 46 76 68]]

 [[ 0 81 47 24  0]
  [ 0 68 87 68 83]
  [86  0 38 94 17]
  [ 0 52 81 99 59]
  [37 69  0 66 68]]

 [[26 81 72 13 92]
  [ 0 73 67 49 21]
  [13 18 36 26 79]
  [66 36  0 12 92]
  [ 0  0 48 44 43]]]
```

In [ ]:

```
1
```